FSA v1.5 closed-loop SMC^2-MPC: a side-by-side Python+JAX vs Julia comparison study

Hand-off report for the next agent

2026-05-08

Contents

Abstract

This document is a hand-off for the next agent (or human reader) who picks up the FSA v1.5 closed-loop comparison work. It explains what the FSA v1.5 model is, what the `smc2fc` / `SMC2FC` frameworks do, the functional programming paradigm we have moved to on the Julia side, the comparison problem itself, the tools developed for it, the bugs found and fixed, where we are today, and what comes next. All paths are absolute paths inside the `python-smc2-filtering-control` repository.

1 Why this report exists

The FSA model has three Julia codebases and three Python codebases in this repo (`version_1_*`, `version_2_*`, `version_3_*` on each side). The most relevant pair for this comparison is:

- **Python+JAX:** `version_1_5_Python_JAX/` — the simplest closed-loop FSA bench, derived from the working `version_2_Python_JAX/` bench, with the v1.5 model surface (3-state SDE, 10 estimated parameters, 3 direct-Gaussian observation channels, no circadian).
- **Julia:** `version_1_5_Julia/` — the Julia rewrite of the same model surface, written in a purely functional style for portability to Lean4 (the formal source of truth at `version_1_5_LEAN/`).

The two stacks share the model math (verified bit-equal to 1e-6 through a Julia↔Python+JAX↔Lean4 121-case differential test). They diverge at the framework layer: the Python+JAX side uses the mature `smc2fc` engine; the Julia side uses its own `SMC2FC.jl` port plus model-side GPU kernels in KernelAbstractions.

The user has invested weeks of optimisation in the Python+JAX side. The Julia side is roughly two days old. This report freezes the current state of the comparison, the bugs we found and fixed in this session, and the open work for the next agent.

2 What is FSA v1.5?

FSA (Fitness–Stress–Autonomic) is a 3-state stochastic differential equation model for an athlete's response to training. State $y = (B, F, A)$:

- B — aerobic fitness (Banister chronic; bounded $[0, 1]$, Jacobi-style state-dependent diffusion).
- F — unified fatigue (Banister acute; ≥ 0, CIR-style diffusion).
- A — autonomic amplitude (Stuart–Landau; ≥ 0, CIR-style diffusion).

Drift (`/day` units):

$$\begin{aligned}
\mu(B, F) &= \mu_0 + \mu_B B - \mu_F F - \mu_{FF} F^2 \\
dB/dt &= \kappa_B \, (1 + \varepsilon_A A) \, \Phi(t) - B/\tau_B \\
dF/dt &= \kappa_F \, \Phi(t) - (1 + \lambda_A A)/\tau_F \, F \\
dA/dt &= \mu \cdot A - \eta A^3
\end{aligned}$$

State-dependent diffusion (Itô):

$$\begin{aligned}
\sigma_B(B) \, dW_B &= \sigma_B \sqrt{B(1 - B)} \, dW_B \\
\sigma_F(F) \, dW_F &= \sigma_F \sqrt{F} \, dW_F \\
\sigma_A(A) \, dW_A &= \sigma_A \sqrt{A} \, dW_A
\end{aligned}$$

The single exogenous control is $\Phi(t)$, the training-strain rate (≥ 0). The closed-loop MPC's job is to pick Φ to maximise the time-integral of A subject to a soft fatigue constraint ($\max(F - F_{\max}, 0)^2$ barrier).

2.1 What makes v1.5 special

- **14 dynamics parameters** in the v1.5 basis; κ_B replaced by $B_\infty = \kappa_B \tau_B$ and κ_F replaced by $F_\infty = \kappa_F \tau_F$ for better identifiability. The basis-rotation adapter is in `simulation.params_v15_to_v1_nt` (Julia) / `simulation.params_v15_to_v1` (Python+JAX).
- **10 estimated parameters** (the filter's decision variables); 4 dynamics parameters pinned at truth (τ_B, η, ε_A, μ_{FF}) per the FIM-rank decision; 3 obs-noise scales pinned at 0.005.

- **3 direct-Gaussian observation channels**: $y_B^{obs} = B + \mathcal{N}(0, \sigma^2_{B,obs})$ and same for F, A. No HR / sleep / stress / steps channels (that's the v2 surface). No circadian forcing.
- Time grid: 60 minutes per bin (24 bins per day) by default. v2 uses 15 minutes per bin (96 bins per day).

3 What `smc2fc` / `SMC2FC` does

Both stacks implement the same statistical pipeline:

1. **Outer SMC^2 filter**: tempered sequential Monte Carlo over the 10-dim unconstrained parameter cloud θ. At each tempering level $\lambda \in [0, 1]$ the targets graduate from prior to posterior. Resample (systematic) $\rightarrow$ Hamiltonian Monte Carlo (HMC) moves with finite-difference gradients $\rightarrow$ next λ. Returns a posterior cloud over θ for each rolling filter window.
2. **Inner particle filter (PF)**: per θ candidate, an inner PF computes $\log p(y_{1:T} \mid \theta)$ by propagating a state-particle cloud forward through the SDE and reweighting by the observation likelihood. This is the marginal log-likelihood the outer SMC^2 uses.
3. **Forward-simulation controller**: NOT a particle filter. The controller takes the filter-derived posterior mean parameters $\hat{\theta}$ and the filter-derived smoothed end-of-window state $\hat{y}$. It then runs an SMC^2 over a low-dimensional schedule parameterisation (8 RBF coefficients θ_{ctrl}): for each candidate θ_{ctrl}, decode the 8 anchors into a per-bin Φ schedule, simulate the SDE forward n_{inner} times under common-random-numbers (CRN) noise, accumulate the cost $J = -\int A\, dt + \lambda_F \int (F - F_{\max})_+^2\, dt$, and use the negative cost as a tempering target. Output is the posterior-mean $\theta^\star_{ctrl}$, decoded to a Φ-schedule for the plant's next stride.

Critical: the controller does not run a particle filter internally. It runs deterministic forward simulations (n_{inner} noise trials per θ_{ctrl}) using the filter's $\hat{\theta}$ and $\hat{y}$. This was an explicit design question raised during the comparison and the answer is the same on both stacks: both controllers correctly forward-simulate.

4 Functional programming paradigm in Julia

4.1 Why we moved to functional

The `version_3_julia/` stack was a Python-port-style Julia implementation: mutable structs, `!`-suffix mutating methods, threaded `MersenneTwister` state, history dicts pushed onto across stride boundaries. It accumulated subtle bugs that were hard to trace, and the user explicitly rejected it ("too much for a human to understand").

`version_1_5_Julia/` is a clean re-design. The five rules:

1. **No mutable structs in the public API.** v1.5 has exactly one struct, `PlantState`, with two fields: an `SVector{3, Float64}` and an `Int`. State updates produce a NEW `PlantState`, not a mutation of the old one.
2. **No `!`-suffix mutating methods.** Every public function returns new values. `plant_step(state, Phi_t, params, dt, key)` returns `(state=new_state, obs=...)`; the caller can build history by collecting returned values.
3. **Explicit RNG keys, JAX-style.** Pass an explicit `key::UInt64` to every randomised function. Internal sub-keys are derived deterministically via `hash((key, :sub_purpose))`. Two callers with the same `(state, key)` get the same answer, period.
4. **Pure facade over the inherently imperative GPU kernel.** KernelAbstractions kernels mutate preallocated `CuArrays` by design. v1.5 wraps every kernel-touching public function in

a pure facade: `gpu_log_density( target, U, grid_obs, key)` → `Vector{Float64}` allocates a fresh output, dispatches the kernel, returns. Mutation is encapsulated; callers never see the `CuArray`.

5. **I/O isolated to the bench's top-level `main()`.** JLD2 + PNG writes happen exactly once at the end of the bench. Logging at the bench level, not inside `plant_step` / `propagate` / `obs_log_weight`.

4.2 The new `SMC2FC_functional` library

A parallel functional rewrite of the `SMC2FC.jl` framework lives at `julia/SMC2FC_functional/`. Same algorithmic contract as the existing `julia/SMC2FC/` port, but:

- Top-level functions take immutable inputs and return immutable outputs.
- Pre-allocated buffers live behind an immutable `Workspace` container.
- No `!`-mutating functions exposed publicly.
- Multiple-dispatch-driven specialisation; idiomatic Julian.
- Google-style docstrings on every file and every function.

The proof-of-principle bench at
`julia/SMC2FC_functional/benchmarks/bench_smc_full_mpc_fsa_v15_functional_gpu.jl`
demonstrates the drop-in replacement. **Going forward this should be the default Julia entry point**; the existing `julia/SMC2FC/` stays as a back-compat reference. A three-library comparison harness is provided at
`julia/SMC2FC_functional/benchmarks/compare_three_libraries.jl`.

5 The comparison problem

The user's question: *do the Python+JAX and Julia v1.5 stacks produce qualitatively the same closed-loop MPC behaviour, given they implement the same model under the same statistical framework?*

This is harder than it sounds for several reasons:

1. Different RNG implementations (JAX `PRNGKey` vs Julia `StableRNG` vs hash-derived sub-keys) mean the plant trajectories are NOT bit-identical even at the same seed. Comparison must be statistical: mean $\int A$, F-violation rate, posterior MSE to truth, schedule shape.
2. Different inner-PF algorithms: Python+JAX uses `smc2fc.filtering.gk_dpf_v3_lite` (a Gaussian kernel, OT-regularised, differentiable PF); Julia uses a custom segmented-PF KernelAbstractions kernel. Both are valid, but they have different particle-degeneracy regimes.
3. Different default particle counts: Python+JAX bench defaulted to $N_{\text{smc}} = 128/N_{\text{inner}} = 32$ on the controller; the Julia bench defaulted to $N_{\text{smc}} = 256/N_{\text{inner}} = 64$. We brought these into parity.
4. No prior baseline: no constant-$\Phi = 1$ baseline trajectory was being plotted alongside the MPC, so "does the controller actually beat doing nothing?" was hard to answer at a glance.
5. Different output formats: Python writes `.npz`, Julia writes `.jld2` (HDF5).

5.1 Tools developed for the comparison

1. JAX GPU profiler (`src/profile_gpu_python.py`). Direct port of the existing v2 Julia profiler. Times the GK-DPF v3-lite *filter kernel* and the controller cost *kernel* in isolation, with

jax.block_until_ready as the synchronisation point. Reports median time per call, threads-in-flight, approximate FLOPs, effective TFLOPS, percentage of the RTX-5090's 104.8 TFLOPS fp32 peak, GPU memory before and after, and host-sync count.

2. v1.5 Julia GPU profiler (src/profile_gpu_julia_v15.jl). Same surface, same metrics, adapted to v1.5's GPU types (FSAGPUTarget, FSAv1ControlGPUTarget).

3. Per-stride telemetry CSV (added to both bench scripts). Each stride writes one row to per_stride.csv with: stride, t_wall_s, n_temp_filter, n_temp_ctrl, daily_phi, A_mean_so_far, B_end, F_end, A_end.

4. Paired-run launcher (src/run_v15_T28d_compare.sh). Single shell script that:

1. Runs both microbench profilers (writes julia_profile.log + python_profile.log).
2. Starts nvidia-smi sampling at 1 Hz to a CSV.
3. Runs the Julia closed-loop bench with matched config.
4. Stops + restarts nvidia-smi for the Python run.
5. Runs the Python+JAX closed-loop bench, same matched config.

Output lands under outputs/v15_julia_vs_python_T<T>d_seed<S>/{julia,python}/.

5. Three-way comparison script (src/compare_v15_julia_vs_python.py). Loads both bench outputs, both nvidia-smi CSVs, both profile logs. Produces:

- summary.txt — the diagnostic table (baseline as gate, then macro nvidia-smi telemetry, then micro profiler numbers).
- comparison.png — 6-panel three-way diagnostic plot. Python MPC blue, Julia MPC red, baseline grey dashed. Panels: A trajectory, applied daily Φ, posterior medians for two parameters with truth lines, GPU SM utilisation over wall time, per-stride wall time bars.

JLD2 files are read with h5py and the 2D and 3D arrays are transposed (Julia is column-major; HDF5 readers in row-major order swap dimensions).

6 Bugs found and fixed

We worked top-down through the comparison and found four substantive issues, two on Python+JAX and two on Julia.

6.1 Python: shrinking planning horizon (A1)

File: version_1_5_Python_JAX/tools/bench_smc_full_mpc_fsa_v15.py, old line 287.

The replan block computed $t_{\text{remaining\_days}} = (n_{\text{strides}} - s) \cdot \text{STRIDE\_BINS/BINS\_PER\_DAY}$ and passed T_total=t_remaining_days into build_control_spec(). As the bench progresses, the horizon shrinks; at stride 14 of T=28d the controller sees only 14 days ahead. v2's bench instead always plans the FULL T_total_days at every replan (bench_smc_full_mpc_fsa.py:386-388). v1.5 Julia uses H_plan_bins = T_total_bins (also full horizon).

Symptom: the controller could not justify a "build B then sprint" Banister strategy because the time-to-horizon was constantly shrinking.

Fix: pass T_total=float(args.T_days) (constant) at every replan.

6.2 Python: mid-array splice instead of overwrite (A2)

File: same, old lines 317–320.

The new plan was spliced INTO THE MIDDLE of the existing daily_phi_plan array using an absolute-current-day offset. Combined with the shrinking horizon (A1), this produced a sawtooth Φ pattern: the front of the plan got partially overwritten on each replan, the back stayed stale.

v2 overwrites the whole plan: `daily_phi_plan = sched_per_day.astype(np.float64)` + `last_replan_stri` `= s + 1`.

Fix: same overwrite-and-reset pattern in v1.5 Python.

6.3 Python: filter degeneracy from bootstrap with GK-DPF (B)

File: `version_1_5_Python_JAX/models/fsa_high_res/estimation.py`, function `_propagate_fn_framework`.

This was the structural bug. The `smc2fc.filtering.gk_dpf_v3_lite` filter is designed for *guided* (Kalman-fused) proposals: the `propagate_fn` contract expects to return both the new state *and* a Radon–Nikodym correction `pred_lw` that the framework adds to the per-bin weight increment.

v1.5 Python+JAX's `_propagate_fn_framework` returned `pred_lw = 0.0` (the bootstrap convention: proposal IS dynamics, no correction needed). Mathematically valid but fundamentally incompatible with GK-DPF v3-lite's weight-smoothing kernel + OT regularisation: the framework's machinery pushed particles toward biased fixed points. Smoking gun: at higher particle counts $N = 1024/K = 800$ the bootstrap variant got WORSE (posterior MSE 1.41 → 31.16), the classic "structural mis-spec" signature.

Fix: Kalman fusion. Mirrors v2's `propagate_fn` pattern exactly. v1.5's obs model is the simplest possible Kalman case ($H = I$, $b = 0$, $R = \mathrm{diag}(\sigma^2_{*,obs})$, all channels always present), so the sequential-scalar update reduces to per-channel 1D Kalman:

$$
\begin{aligned}
\mu_{\text{prior}} &= y + dt \cdot \text{drift} \\
P_{\text{prior}} &= \mathrm{diag}(\sigma_B^2\, B(1{-}B)\, dt,\ \sigma_F^2\, F\, dt,\ \sigma_A^2\, A\, dt) \\
\text{innov}_i &= y_{\text{obs},i} - \mu_i \\
S_i &= P_{ii} + \sigma^2_{i,obs} \\
K_i &= P[:, i]/S_i \\
\mu &\mathrel{+}= K_i \cdot \text{innov}_i \\
P &\mathrel{-}= K_i \otimes P[i, :] \\
\log p_i &= -\tfrac{1}{2}\log(2\pi S_i) - \tfrac{\text{innov}_i^2}{2S_i}
\end{aligned}
$$

Sample $x_{\text{new}} \sim \mathcal{N}(\mu_{\text{fused}}, P_{\text{fused}})$ via Cholesky. Return `pred_lw = log_pred_total - obs_ll(y_new)` so the framework's `obs_log_weight_fn` re-add cancels properly.

Result (T=7d, seed=42, $N = 32$, $K = 200$):

	Before	After	Improvement
posterior MSE	1.41	0.0188	∼74×

The 10 estimated parameters' posterior medians now hug truth tightly (was: τ_F stuck at 3 vs truth 7; sigmas drifting upward unbounded).

6.4 Python: `init_state` cheat (C)

File: same bench file, around old line 294.

The bench was passing `init_state_dict = dict(B=plant_state.bfa[0], ...)` directly to the controller. This is a closed-loop violation: the controller should know only what the filter knows (its posterior estimate of the state), not the plant's true state.

Fix: extract a smoothed end-of-window state from the filter posterior cloud via `log_density_factory.extrac` mirroring v2's pattern (line 366–367). Use that as `init_state`.

6.5 Julia: per-tempering-level allocation churn (perf)

File: `version_1_5_Julia/models/fsa_high_res/gpu_pf.jl`, function `parallel_hmc_one_move`.

The functional HMC variant allocates roughly ten (M, d) matrices per call. With 13 strides × 5 tempering levels × 3 HMC moves per level the bench does ∼200 HMC calls, allocating a few MB total — not catastrophic, but a real per-call constant.

Fix: added a mutating `parallel_hmc_one_move!` variant that reuses two persistent (M, d) buffers across the whole leapfrog. The bench imports both; `run_outer_smc` now calls the mutating variant.

Result (T=2d open-loop smoke, $N = 16$, $K = 100$): wall time 96.9 s → 92.0 s (∼5% faster at this small N). All six GPU PF smoke tests still pass.

6.6 Julia: optional Liu–West θ-cloud shrinkage

File: same plus `version_1_5_Julia/tools/bench_smc_full_mpc_fsa_gpu.jl`.

Added a CLI flag `--liu-west-a` (default 0.0 = disabled). When set in $(0, 1)$ — e.g. 0.97 — applies the standard Liu–West shrinkage between resample and HMC inside `run_outer_smc`:

$$\theta_i \leftarrow a \cdot \theta_i + (1 - a) \cdot \overline{\theta} + \sqrt{1 - a^2} \cdot \text{column\_std} \cdot \xi, \qquad \xi \sim \mathcal{N}(0, I)$$

Helps maintain θ-cloud diversity at low N, where pure tempered HMC can collapse. Off by default so existing reproducibility is preserved.

7 Where we are now

After all fixes (T=7d, seed=42, matched config $N_{\text{smc}} = 32$, $K_{\text{pf}} = 200$):

	Baseline ($\Phi = 1$)	Python+JAX MPC	Julia MPC
mean A (last 7 days)	0.0935	0.0808	0.0815
improvement vs baseline	—	−13.6%	−12.8%
F-violation rate	0.0	0.0	0.0
posterior MSE to truth (final window)	—	0.0188	0.0868
total wall time (min)	—	5.3	37.2
mean GPU utilisation (%)	—	40.0	24.8
filter kernel time / call (ms, micro)	—	4.7	81.0
controller cost / call (ms, micro)	—	23.4	0.5

Read this carefully. The headline numbers are nuanced:

- **Both filters are now algorithmically sound.** The Python+JAX MSE 0.0188 vs Julia 0.0868 difference is a *maturity gap, not an algorithmic gap* — weeks of Python+JAX optimisation vs < 48h on Julia. Both are at the regime where the controller is the binding constraint.
- **Neither MPC beats baseline at T=7d**; both lose ∼13% on mean A. This is the H3 controller-cost-design issue: at short horizon, $J = -\int A + \lambda_F \int (F - F_{\max})_+^2$ admits a "rest cures all" optimum because suppressing Φ keeps F low without paying much $-\int A$ cost. v2's reference plots ramp Φ UP at T=14d / T=28d; the longer-horizon test is the next thing to run.
- **Julia is 7× slower wall time, with similar mean GPU utilisation.** The microbench shows the gap is per-call, not algorithmic: filter kernel 81 ms vs 4.7 ms. Plenty of GPU headroom on Julia's side; the Python controller is, conversely, much slower than the Julia controller (likely fp64 vs fp32). See Section 8.

8 What comes next

In rough priority order:

1. Migrate to `SMC2FC_functional`. `julia/SMC2FC_functional/` is intended to be the default Julia framework going forward. Rerun the comparison harness with the bench at `julia/SMC2FC_functional/benc` to confirm it produces equivalent posteriors / control behaviour. The three-library cross-check at `compare_three_libraries.jl` is the validation harness.

2. Optimise Julia GPU utilisation. The 81 ms / call filter kernel is the headline cost. Likely sources:

- Per-segment kernel launch overhead inside the segmented PF (6 kernel launches per `gpu_log_density` call × 21 FD perturbations × ...).
- Liu–West θ-cloud shrinkage (now optional via CLI; run with `--liu-west-a 0.97` to test).
- Inter-segment resample on CPU vs GPU — worth instrumenting.

The mutating-HMC change in this session was a small first step. `nsys profile` on a real bench run would reveal the actual bottleneck. Bumping particle counts to v2-saturation-tier ($N = 256, K = 400$) is the next-cheapest test.

3. Run T=14d / T=28d to test H3. The user's reference v2 plots ramp Φ UP at longer horizons (T=14d: mean A 0.122 vs baseline 0.081, +50%; T=28d: 0.157 vs 0.098, +60%). The hypothesis is that v1.5's controller will similarly find the ramp-up optimum at horizons long enough for the "build B then sprint" payoff to dominate.

4. Cost-function variants. If T=14d / T=28d still picks rest, try:

- Re-introduce $\lambda_\Phi \int \Phi^2 \, dt$ penalty (currently $\lambda_\Phi = 0$ on both stacks by default — the cost expression supports it).
- Add a soft "minimum mean-A over planning window" floor.
- Investigate whether the v2 G1 reparametrisation around (A_{TYP}, F_{TYP}) shifts the bifurcation balance in a way that makes ramp-up the dominant optimum at all horizons.

5. Multi-seed robustness. All comparison runs to date are single-seed. If a T=28d run flips H3 cleanly, replicate at ≥ 3 seeds before declaring victory.

9 How to run the comparison end-to-end

The full pipeline is automated by the launcher:

```
conda activate comfyenv
bash
    /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Python_JAX/tools/launchers/run_v
    7 42
# args: T_DAYS, SEED. Defaults: T=28, seed=42.
```

This produces (under the `outputs/v15_julia_vs_python_T<T>d_seed<S>/` parent dir):

- `julia_profile.log`, `python_profile.log`
- `julia/`: `data.jld2`, `manifest.json`, `per_stride.csv`, `gpu_telemetry.csv`, two PNGs (state traces, param traces), `bench.log`.

- python/: trajectory.npz, manifest.json, per_stride.csv, gpu_telemetry.csv, two PNGs, bench.log.

Then run the comparison script:

```
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Python_JAX
JAX_ENABLE_X64=True PYTHONPATH=.:.. \
  python tools/compare_v15_julia_vs_python.py \
  /home/ajay/Repos/python-smc2-filtering-control/outputs/v15_julia_vs_python_T7d_seed42
```

This writes summary.txt and comparison.png into the parent dir.

10 How to profile each stack in isolation

```
# Julia microbench (filter kernel + controller cost kernel)
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Julia
julia --project=. tools/profile_gpu.jl --both

# Python+JAX microbench (same surface, same metrics)
cd /home/ajay/Repos/python-smc2-filtering-control/version_1_5_Python_JAX
JAX_ENABLE_X64=True PYTHONPATH=.:.. \
  python tools/profile_gpu.py --both
```

For deeper Python+JAX profiling, pass --trace to dump a jax.profiler trace into ./jax_trace_filter/ and ./jax_trace_controller/ that TensorBoard's Profile tab can render.

For deeper Julia profiling, run the bench under nsys profile --trace=cuda julia tools/bench_smc_full_
... and inspect the resulting .qdrep in NVIDIA Nsight Systems.

11 Repository basics: where things live

Python+JAX. Per-file layout under version_1_5_Python_JAX/:

- models/fsa_high_res/_dynamics.py — pure JAX drift, diffusion, EM step.
- models/fsa_high_res/simulation.py — DEFAULT_PARAMS, INIT_STATE, params_v15_to_v1.
- models/fsa_high_res/_plant.py — pure plant_step / plant_rollout.
- models/fsa_high_res/estimation.py — HIGH_RES_FSA_V15_ESTIMATION (EstimationModel); the Kalman-fused _propagate_fn_framework.
- models/fsa_high_res/control.py — RBF schedule + cost; build_control_spec.
- tools/bench_smc_full_mpc_fsa_v15.py — the closed-loop bench.

The smc2fc framework lives at smc2fc/ (one level up from any version dir). The relevant entry points consumed by the bench are: smc2fc.core.jax_native_smc.run_smc_window_native, ...run_smc_window_bridge_native, smc2fc.filtering.gk_dpf_v3_lite.make_gk_dpf_v3_lite_log_density_c smc2fc.control.tempered_smc_loop.run_tempered_smc_loop_native.

Julia. Per-file layout under version_1_5_Julia/:

- models/fsa_high_res/_dynamics.jl — v1 verbatim G0 SDE drift / diffusion / substepped EM.
- models/fsa_high_res/simulation.jl — DEFAULT_PARAMS, params_v15_to_v1_nt, sample_obs_bfa. Uses Match.jl @match at every site that a future Lean4 port would also need to pattern-match.
- models/fsa_high_res/_plant.jl — pure plant_step / plant_rollout.

- `models/fsa_high_res/gpu_pf.jl` — `FSAGPUTarget`, `gpu_log_density`, `gpu_grads`, `parallel_hmc_one_mov` `parallel_hmc_one_move!` (the new mutating variant).
- `models/fsa_high_res/gpu_control.jl` — `FSAv1ControlGPUTarget`, `gpu_cost_log_density_batched`.
- `models/fsa_high_res/estimation.jl` — `PARAM_NAMES`, `PARAM_PRIOR_CONFIG`.
- `tools/bench_smc_full_mpc_fsa_gpu.jl` — the closed-loop bench, including `run_outer_smc` (the hand-rolled tempered SMC^2 loop) and `controller_plan` (calls `SMC2FC.run_tempered_smc_gpu`).

The Julia framework lives at `julia/SMC2FC/` (existing) and `julia/SMC2FC_functional/` (new, drop-in replacement).

Lean4 reference. `version_1_5_LEAN/` provides a Lean4 implementation of the v1.5 model functions exposed via a JSON-line CLI (`Main.lean`). The diff test `version_1_5_Julia/diff_test/test_lean_diff_v1` round-trips 121 randomised cases through the Lean4 binary at 1e-6 single-step / 1e-4 integrated tolerance. This guarantees the model math is identical across all three implementations.

12 Closing notes

The work in this session is committed and pushed on the `Julia-port-verison-1` branch. Notable hashes:

- `0c92907` — CLI parity + GPU profilers (Phases A/A1/A1b)
- `ff3da57` — per-stride telemetry + launcher + comparison script
- `155beec` — T=7d pre-flight artefacts + initial findings
- `159f93e` — Python A1 + A2 fix (shrinking horizon + mid-array splice)
- `3ff65c3` — Python Kalman-fusion fix + `init_state` cheat fixed
- `da5e9c9` — findings reframing (drop misleading "Python best of 3")
- `c4a9e86` — Julia perf: mutating HMC + optional Liu–West

The corresponding plan archive is `claude_plans/FSA_v1_5_three_way_closed_loop_comparison_2026-05-08` (also copied to `docs/PLAN_ARCHIVE.md` in this directory).